

UNIVERSIDAD DE GUADALAJARA

Centro Universitario de Ciencias Exactas e Ingenierías (CUCEI)

División de Electrónica y Computación



Arquitectura de Computadoras

Verano 2026

MARCO EDUARDO GONZALEZ LUA

OSCAR EMMANUEL QUEZADA CAMACHO

1. Introducción

El estudio de la arquitectura de computadoras tiene como uno de sus pilares fundamentales la comprensión de cómo un conjunto de instrucciones (ISA, por sus siglas en inglés) se traduce en una implementación física de hardware capaz de ejecutarlo. La arquitectura MIPS, por su carácter didáctico, su repertorio de instrucciones reducido (RISC) y su codificación uniforme de 32 bits, constituye uno de los modelos de referencia más utilizados en la enseñanza del diseño de procesadores, ya que permite abordar de forma progresiva conceptos como la ruta de datos (datapath), la unidad de control, la segmentación de instrucciones (pipelining) y el manejo de riesgos estructurales, de datos y de control.

Un procesador segmentado divide el procesamiento de cada instrucción en varias etapas independientes —tradicionalmente Búsqueda de Instrucción (IF), Decodificación (ID), Ejecución (EX), Acceso a Memoria (MEM) y Escritura de Resultados (WB)— de modo que, en un instante dado, varias instrucciones distintas se encuentran avanzando simultáneamente por etapas diferentes del procesador. Esta técnica incrementa de manera significativa el rendimiento (throughput) del sistema, a costa de introducir una mayor complejidad de control, ya que es necesario resguardar el estado de cada instrucción entre etapas mediante registros de segmentación (pipeline registers) y, en diseños completos, gestionar los riesgos de segmentación que surgen de las dependencias entre instrucciones consecutivas.

En el presente reporte se documenta el diseño, la descripción en lenguaje Verilog y la validación por simulación de un datapath MIPS de 32 bits segmentado en cinco etapas, capaz de decodificar y ejecutar instrucciones de los tres formatos fundamentales de la arquitectura: Tipo R (operaciones aritmético-lógicas entre registros, como add, sub, and, or y slt), Tipo I (instrucciones con inmediato y acceso a memoria, como addi, andi, ori, sli, lw, sw y beq) y Tipo J (salto incondicional j). El diseño se integra a partir de módulos independientes —unidad de control, banco de registros, ALU, unidad de control de la ALU, extensor de signo, multiplexores, memorias de instrucciones y de datos, y cuatro registros de segmentación— y se valida mediante un programa de

prueba en ensamblador que realiza la búsqueda del valor máximo dentro de un arreglo y el conteo de elementos impares, cuyo comportamiento se contrasta con las formas de onda obtenidas en el simulador ModelSim.

El documento se organiza de la siguiente manera: en la sección de Desarrollo se describe la arquitectura general del datapath, el propósito de cada módulo, las señales de control generadas para cada tipo de instrucción, la codificación de la unidad ALUControl, el programa de prueba utilizado y su correspondencia con el código máquina, los resultados esperados de la simulación y, finalmente, un análisis de las consideraciones y limitaciones identificadas en torno al manejo de riesgos de segmentación. Posteriormente se presentan las conclusiones obtenidas y las referencias bibliográficas consultadas.

2. Objetivo General

Diseñar, describir en lenguaje Verilog e implementar un datapath MIPS de 32 bits segmentado en cinco etapas (IF, ID, EX, MEM y WB), capaz de decodificar y ejecutar correctamente instrucciones de los tres formatos fundamentales de la arquitectura MIPS —Tipo R, Tipo I y Tipo J—, integrando la unidad de control, el banco de registros, la unidad aritmético-lógica, las memorias de instrucciones y de datos, y los registros de segmentación correspondientes, y validar su funcionamiento mediante la simulación de un programa de prueba representativo.

3. Objetivos Particulares

- Implementar la unidad de control (módulo CONTROL) que genere, a partir del campo opcode de la instrucción, las señales RegDst, ALUSrc, MemToReg, RegWrite, MemRead, MemWrite, Branch, Jump y ALUOp.
- Diseñar el banco de registros (BancoREG) de 32 registros de 32 bits, con dos puertos de lectura combinacional y un puerto de escritura sincronizada al flanco de reloj, incluyendo el bypass de escritura-lectura en el mismo ciclo.

- Implementar la Unidad Aritmético-Lógica (ALU) y su unidad de control asociada (ALUCONTROL), capaces de ejecutar las operaciones AND, OR, ADD, SUB y SLT, así como generar la bandera Zero.
- Construir los cuatro registros de segmentación (IF/ID, ID/EX, EX/MEM y MEM/WB) que almacenan y propagan tanto los datos como las señales de control entre etapas consecutivas del pipeline.
- Implementar la memoria de instrucciones (MeMInst) y la memoria de datos (ARAM), ambas inicializadas a partir de archivos externos en formato binario.
- Incorporar la lógica de resolución de saltos condicionales (BEQ) e incondicionales (J), incluyendo el cálculo de las direcciones destino correspondientes.
- Integrar la totalidad de los módulos en un datapath completo (MIPS.v), generando la señal de reloj y el entorno de simulación necesarios para su verificación.
- Verificar el funcionamiento del procesador mediante un programa de prueba en ensamblador (búsqueda del valor máximo y conteo de números impares en un arreglo) y el análisis de las formas de onda resultantes en el simulador ModelSim.

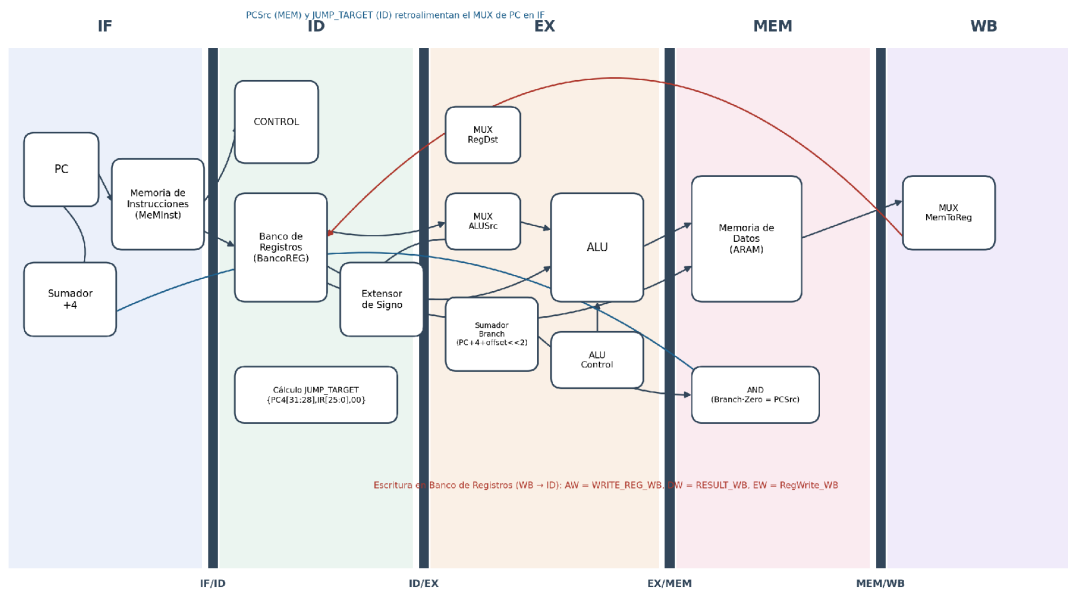
4. Desarrollo

4.1 Arquitectura general del datapath

El procesador se organiza en cinco etapas clásicas de un pipeline MIPS, separadas por cuatro registros de segmentación que retienen, en cada flanco de reloj, tanto los datos como las señales de control que la instrucción necesitará en etapas posteriores:

- IF (Instruction Fetch): obtiene la instrucción de la memoria de instrucciones a partir del valor actual del Program Counter (PC) y calcula PC+4.
- ID (Instruction Decode): decodifica el opcode mediante la unidad de control, lee los registros fuente del banco de registros, extiende de signo el inmediato y calcula la dirección absoluta de salto (JUMP_TARGET).
- EX (Execute): selecciona los operandos de la ALU mediante los multiplexores RegDst y ALUSrc, ejecuta la operación indicada por ALUControl y calcula la dirección destino del branch ($PC+4 + \text{desplazamiento} \times 4$).
- MEM (Memory Access): accede a la memoria de datos (lectura para lw, escritura para sw) y resuelve el salto condicional BEQ mediante la compuerta AND entre las señales Branch y Zero (PCSrc).
- WB (Write Back): selecciona, mediante el multiplexor MemToReg, entre el dato leído de memoria y el resultado de la ALU, y lo escribe de regreso en el banco de registros.

La Figura 1 presenta el diagrama de bloques general del datapath, mostrando la disposición de los módulos en cada etapa, los registros de segmentación (IF/ID, ID/EX, EX/MEM y MEM/WB) y los dos caminos de retroalimentación propios de un pipeline con saltos: la escritura del banco de registros desde la etapa WB hacia ID, y la señal PCSrc/JUMP_TARGET que retroalimenta el multiplexor de selección del PC en la etapa IF.



4.2 Descripción de los módulos del datapath

La Tabla 1 resume la función de cada módulo Verilog que conforma el datapath, agrupados según la etapa del pipeline a la que pertenecen principalmente.

Módulo	Archivo	Función principal
PC	PC.v	Registro de 32 bits que almacena la dirección de la instrucción actual; se actualiza en cada flanco de subida del reloj.
Sumador (ADDER)	ADDER.v	Sumador genérico parametrizable en ancho; se reutiliza para calcular PC+4 y la dirección destino del branch.
Memoria de Instrucciones	MeMInst.v	Memoria de solo lectura de 256 palabras de 32 bits, inicializada desde Instrucciones.txt y direccionada por Dir[9:2].
IF/ID	IF_ID.v	Registro de segmentación que retiene PC+4 y la instrucción obtenida entre las etapas IF e ID.
CONTROL	CONTROL.v	Decodificador combinacional del opcode que genera las nueve señales de control del datapath.
Banco de Registros	BancoREG.v	32 registros de 32 bits; dos puertos de lectura combinacional (con bypass a la escritura del mismo ciclo) y un puerto de escritura síncrono.
Extensor de Signo	SIGNEXTEND.v	Extiende el inmediato de 16 a 32 bits preservando el bit de signo.
MUX	MUX.v	Multiplexor genérico parametrizable en ancho de datos; se reutiliza en RegDst, ALUSrc, la selección del PC (branch/jump) y MemToReg.

Módulo	Archivo	Función principal
ID/EX	ID_EX.v	Registro de segmentación que propaga datos (PC+4, RD1, RD2, inmediato extendido, campos rs/rt/rd/funct/opcode) y señales de control hacia EX.
ALU Control	ALUCONTROL.v	Traduce ALUOp junto con funct u opcode a la señal de 4 bits que gobierna la operación realizada por la ALU.
ALU	ALU.v	Unidad aritmético-lógica de 32 bits (AND, OR, ADD, SUB, SLT con signo) que además genera la bandera Zero.
EX/MEM	EX_MEM.v	Registro de segmentación que propaga el resultado de la ALU, RD2, el registro destino, la bandera Zero y la dirección de branch hacia MEM.
Memoria de Datos (ARAM)	ARAM.v	Memoria de datos de 256 palabras, con escritura síncrona y lectura combinacional, inicializada desde Memoria.txt.
MEM/WB	MEM_WB.v	Registro de segmentación que propaga el dato leído de memoria y el resultado de la ALU hacia WB.
MIPS (top level)	MIPS.v	Módulo de nivel superior que instancia e interconecta todos los bloques anteriores, genera el reloj de simulación y resuelve los saltos BEQ y J.

Tabla 1. Módulos que integran el datapath y su función principal.

4.3 Señales de control por tipo de instrucción

El módulo CONTROL decodifica de forma combinacional el campo opcode (INSTR[31:26]) y genera las señales que gobiernan al resto del datapath. La Tabla 2 resume la codificación implementada para cada instrucción soportada.

OpCode	Instrucción	RegDs t	ALUSr c	MemToR eg	RegWrit e	MemRea d	MemWrit e	Branc h	Jump	ALUOp
000000	R-Type	1	0	0	1	0	0	0	0	10
001000	ADDI	0	1	0	1	0	0	0	0	00
001100	ANDI	0	1	0	1	0	0	0	0	11
001101	ORI	0	1	0	1	0	0	0	0	11
001010	SLTI	0	1	0	1	0	0	0	0	11
100011	LW	0	1	1	1	1	0	0	0	00
101011	SW	0	1	—	0	0	1	0	0	00
000100	BEQ	0	0	—	0	0	0	1	0	01
000010	J	—	—	—	0	0	0	0	1	—

4.4 Codificación de la unidad ALUControl

A partir de la señal ALUOp (proveniente de CONTROL) y, según el caso, del campo funct (instrucciones Tipo R) o del opcode (instrucciones I-Type especiales), el módulo ALUCONTROL genera el código de 4 bits que selecciona la operación interna de la ALU, conforme a la Tabla 3.

ALUOp	Origen	Operación	Código ALC
00	ADDI / LW / SW	Suma ($A + B$)	0010
01	BEQ	Resta ($A - B$), usada para la bandera Zero	0110
10 · funct 100000	R-Type: ADD	Suma	0010
10 · funct 100010	R-Type: SUB	Resta	0110
10 · funct 100100	R-Type: AND	AND bit a bit	0000
10 · funct 100101	R-Type: OR	OR bit a bit	0001
10 · funct 101010	R-Type: SLT	Comparación con signo ($A < B$)	0111
11 · opcode 001100	ANDI	AND bit a bit	0000
11 · opcode 001101	ORI	OR bit a bit	0001
11 · opcode 001010	SLTI	Comparación con signo ($A < B$)	0111

Tabla 3. Codificación de la señal ALUControl (ALC) implementada en el módulo ALUCONTROL.

4.5 Registros de segmentación

Cada uno de los cuatro registros de segmentación se implementó como un bloque secuencial (always @(posedge clk)) que, en cada ciclo de reloj, captura a su entrada los datos y señales de control producidos por la etapa anterior y los expone a su salida durante todo el siguiente ciclo, permitiendo que varias instrucciones avancen simultáneamente por etapas distintas del pipeline:

- IF/ID: propaga PC+4 y la instrucción completa de 32 bits.
- ID/EX: propaga PC+4, RD1, RD2, el inmediato extendido de signo, los campos rs, rt, rd, funct y opcode, y las nueve señales de control (RegDst, ALUSrc, ALUOp, MemRead, MemWrite, RegWrite, MemToReg y Branch).

- EX/MEM: propaga el resultado de la ALU, RD2 (dato a escribir en memoria), el registro destino, la bandera Zero, la dirección destino del branch y las señales de control MemRead, MemWrite, RegWrite, MemToReg y Branch.
- MEM/WB: propaga el dato leído de memoria, el resultado de la ALU, el registro destino y las señales de control RegWrite y MemToReg.

4.6 Resolución de saltos condicionales e incondicionales

El salto condicional BEQ se resuelve en la etapa MEM: en la etapa EX la ALU calcula $RS - RT$ ($ALUOp = 01$) para producir la bandera Zero, y en paralelo un sumador dedicado calcula la dirección destino como $PC4 +$ (inmediato extendido desplazado dos bits a la izquierda). Ya en la etapa MEM, la señal PCSrc se obtiene como la conjunción (AND) de Branch y Zero, y controla el multiplexor que en la etapa IF selecciona entre $PC+4$ (flujo secuencial) y la dirección destino del branch.

El salto incondicional J, en cambio, se resuelve directamente en la etapa ID: la dirección destino (JUMP_TARGET) se forma concatenando los cuatro bits más significativos de $PC+4$ con los 26 bits de dirección de la instrucción y dos bits en cero ($\{PC4[31:28], INSTR[25:0], 2'b00\}$), y la señal Jump controla de forma combinacional el multiplexor final que alimenta al PC. Al resolverse en una etapa más temprana que el BEQ, el salto J introduce una penalización de segmentación menor que la del BEQ.

4.7 Programa de prueba en ensamblador

Para validar el datapath se utilizó un programa en ensamblador MIPS32 que recorre un arreglo de cinco elementos almacenado en la memoria de datos a partir de la dirección 0, determina el valor máximo del arreglo y cuenta cuántos de sus elementos son impares, guardando ambos resultados en las direcciones 40 y 44 de la memoria de datos. El programa ejercita instrucciones Tipo R (slt, add), Tipo I (addi, andi, lw, sw, beq) y Tipo J (j), cubriendo así la totalidad de los formatos soportados por el datapath.

```
addi $t0, $zero, 0    # $t0 = dirección base del arreglo
addi $t1, $zero, 5    # $t1 = tamaño del arreglo
addi $t2, $zero, 0    # $t2 = contador i = 0
addi $s0, $zero, 0    # $s0 = Máximo = 0
addi $s1, $zero, 0    # $s1 = contador de impares = 0
ciclo:
slt  $t3, $t2, $t1    # $t3 = 1 si (i < tamaño)
beq  $t3, $zero, fin  # si i >= tamaño, sale del ciclo
```

```

lw $t4, 0($t0)    # $t4 = arreglo[i]
andi $t5, $t4, 1  # $t5 = bit menos significativo de $t4
beq $t5, $zero, skip # si es par, no incrementa el contador
addi $s1, $s1, 1   # si es impar, Impares++
skip:
slt $t5, $s0, $t4  # $t5 = 1 si (Máximo < elemento actual)
beq $t5, $zero, next # si no es mayor, no actualiza el máximo
add $s0, $zero, $t4 # Máximo = elemento actual
next:
addi $t0, $t0, 4   # avanza el puntero al siguiente elemento
addi $t2, $t2, 1   # i++
j ciclo            # regresa al inicio del ciclo
fin:
sw $s0, 40($zero) # guarda el Máximo en memoria[40]
sw $s1, 44($zero) # guarda el conteo de Impares en memoria[44]

```

La correspondencia entre cada instrucción, su dirección de memoria y su codificación en binario (contenido de Instrucciones.txt, cargado en el módulo MeMInst) se presenta en la Tabla 4, verificada instrucción por instrucción a partir de la codificación del conjunto de instrucciones MIPS32.

Dir.	Instrucción (ensamblador)	Código máquina (32 bits)
0	addi \$t0, \$zero, 0	00100000000010000000000000000000
4	addi \$t1, \$zero, 5	00100000000010010000000000000101
8	addi \$t2, \$zero, 0	00100000000010100000000000000000
12	addi \$s0, \$zero, 0	00100000000010000000000000000000
16	addi \$s1, \$zero, 0	00100000000010001000000000000000
20	ciclo: slt \$t3, \$t2, \$t1	00000001010010010101100000101010
24	beq \$t3, \$zero, fin	00010001011000000000000000001010
28	lw \$t4, 0(\$t0)	10001101000011000000000000000000
32	andi \$t5, \$t4, 1	00110001100011010000000000000001
36	beq \$t5, \$zero, skip	00010001101000000000000000000001
40	addi \$s1, \$s1, 1	00100010001100010000000000000001
44	skip: slt \$t5, \$s0, \$t4	00000010000011000110100000101010
48	beq \$t5, \$zero, next	00010001101000000000000000000001
52	add \$s0, \$zero, \$t4	00000000000011001000000000100000
56	next: addi \$t0, \$t0, 4	00100001000010000000000000000100
60	addi \$t2, \$t2, 1	00100001010010100000000000000001
64	j ciclo	00001000000000000000000000000101

Dir.	Instrucción (ensamblador)	Código máquina (32 bits)
68	fin: sw \$s0, 40(\$zero)	1010110000010000000000000000101000
72	sw \$s1, 44(\$zero)	1010110000010001000000000000101100

4.8 Datos iniciales de la simulación

La memoria de datos (ARAM) se inicializa a partir del archivo Memoria.txt con el arreglo de cinco elementos que procesa el programa de prueba, mostrado en la Tabla 5. El banco de registros (BancoREG) se inicializa, a su vez, a partir del archivo Banco.txt (Tabla 6); dado que el programa de prueba inicializa explícitamente los registros que utiliza mediante instrucciones `addi` a partir de `$zero`, estos valores iniciales del banco de registros no intervienen en el resultado final de este programa en particular, pero establecen el estado de partida del procesador.

[illegible][illegible]

4.9 Resultados esperados y verificación funcional

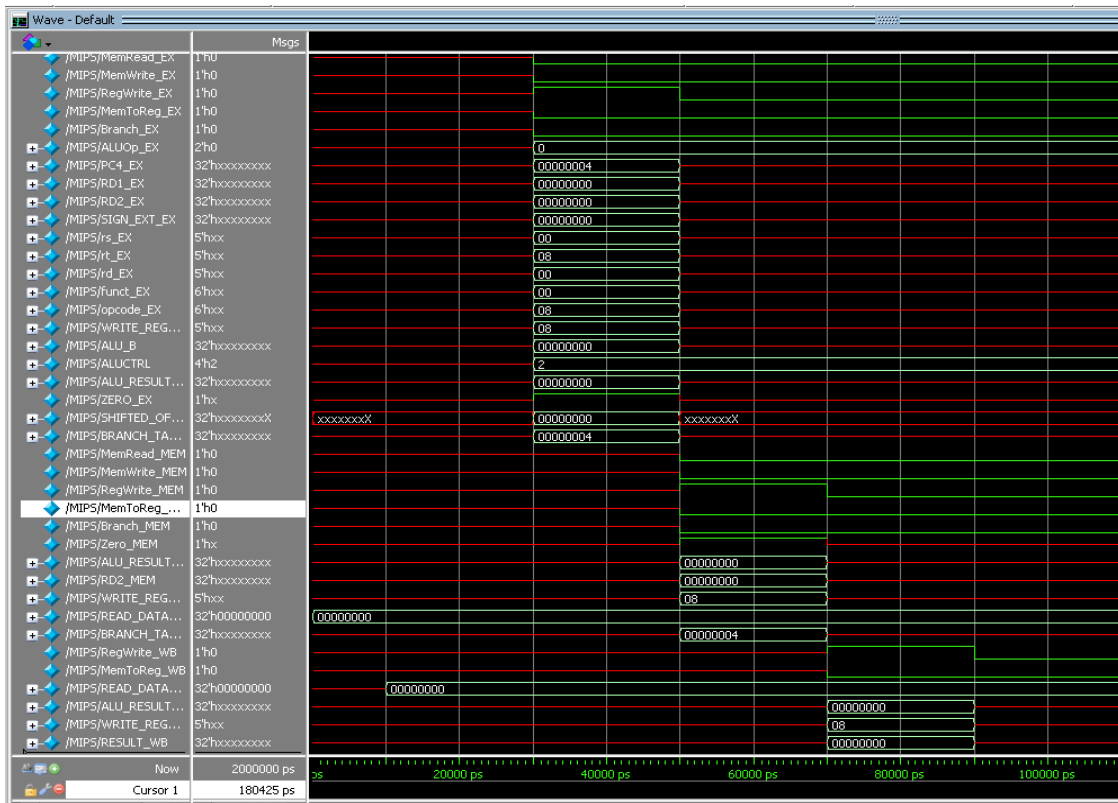
A partir del arreglo de entrada [15, 89, 4, 23, 10] es posible calcular de forma independiente el resultado esperado del programa, para contrastarlo posteriormente contra el comportamiento simulado del datapath:

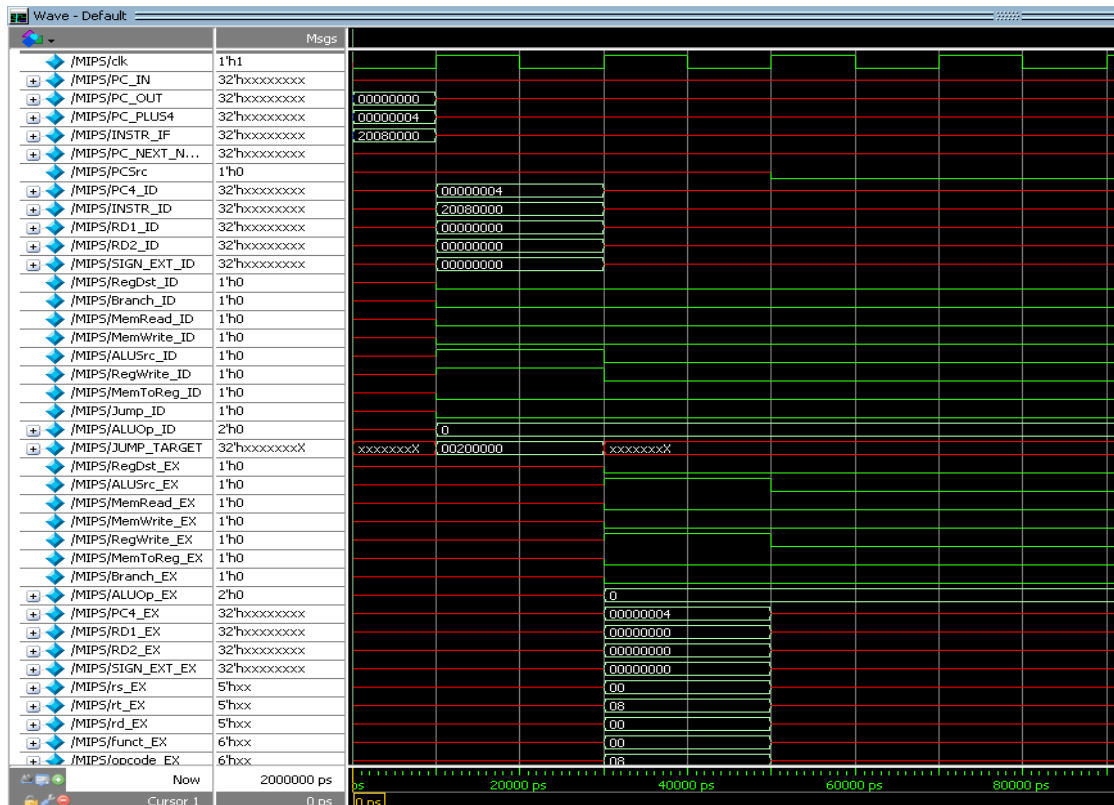
- Paridad de cada elemento: 15 (impar), 89 (impar), 4 (par), 23 (impar), 10 (par) → 3 elementos impares.
- Valor máximo del arreglo: 89.

En consecuencia, al finalizar la ejecución del programa se espera que la memoria de datos contenga, en la dirección 40, el valor 89 (0x00000059) correspondiente al Máximo (\$s0), y en la dirección 44, el valor 3 (0x00000003) correspondiente al conteo de números impares (\$s1).

4.10 Formas de onda de simulación (ModelSim)

El comportamiento del datapath se verificó mediante simulación en ModelSim, observando la propagación de instrucciones y señales a través de las cinco etapas del pipeline. La Figura 2 muestra las señales de las etapas IF e ID durante los primeros ciclos de reloj: se observa cómo, mientras el PC avanza de 0x00000000 a 0x00000004, la instrucción capturada por el registro IF/ID (INSTR_ID = 0x20080000, correspondiente a `addi $t0, $zero, 0`) es decodificada en la misma etapa en que una nueva instrucción ya está siendo obtenida de memoria, evidenciando el solapamiento característico de la ejecución segmentada.





5. Conclusiones

El desarrollo de este proyecto permitió diseñar, describir en Verilog e integrar un datapath MIPS de 32 bits segmentado en cinco etapas, capaz de decodificar y ejecutar correctamente instrucciones de los tres formatos fundamentales de la arquitectura MIPS: Tipo R, Tipo I y Tipo J. La separación modular entre la unidad de control, la ruta de datos y los registros de segmentación facilitó tanto la implementación como la verificación independiente de cada bloque, y permitió comprender de manera práctica cómo la codificación de una instrucción de 32 bits se traduce, señal por señal, en el comportamiento observado en el hardware.

La construcción de la unidad de control y de la unidad ALUControl evidenció la relación directa entre el repertorio de instrucciones de la arquitectura y la lógica combinacional necesaria para decodificarlo, mientras que la implementación de los cuatro registros de segmentación permitió constatar, mediante las formas de onda obtenidas en ModelSim, el solapamiento real de varias instrucciones ejecutándose de forma simultánea en

distintas etapas del procesador —el principio fundamental que justifica el uso de segmentación en el diseño de procesadores modernos.

El uso de un programa de prueba representativo, que combina instrucciones aritméticas, lógicas, de comparación, de acceso a memoria y de salto condicional e incondicional, resultó fundamental para validar de manera integral el datapath y, a la vez, permitió identificar de forma objetiva las limitaciones actuales del diseño en cuanto al manejo de riesgos de segmentación: la ausencia de una unidad de forwarding y de una unidad de detección de riesgos para el caso de carga-uso (load-use), así como la falta de lógica de vaciado (flush) ante saltos condicionales tomados. Ambas observaciones no restan valor al trabajo realizado, sino que delimitan con precisión las siguientes etapas naturales de desarrollo del procesador.

En conjunto, el proyecto cumplió su propósito académico principal: consolidar, mediante el diseño práctico de un procesador segmentado en Verilog, los conocimientos de organización y arquitectura de computadoras relativos a la relación entre el conjunto de instrucciones, la ruta de datos y el control, así como introducir de forma directa los conceptos y retos propios de la segmentación de instrucciones, sentando una base sólida para la incorporación de forwarding, detección de riesgos y otras técnicas de mejora del rendimiento en trabajos posteriores.

6. Referencias

- Harris, D. M., & Harris, S. L. (2012). Digital Design and Computer Architecture: MIPS Edition (2.^a ed.). Morgan Kaufmann.
- Hennessy, J. L., & Patterson, D. A. (2019). Computer Architecture: A Quantitative Approach (6.^a ed.). Morgan Kaufmann.
- Patterson, D. A., & Hennessy, J. L. (2021). Computer Organization and Design: The Hardware/Software Interface, MIPS Edition (5.^a ed.). Morgan Kaufmann.
- MIPS Technologies, Inc. (2001). MIPS32 Architecture for Programmers, Volume II: The MIPS32 Instruction Set.
- IEEE. (2005). IEEE Standard for Verilog Hardware Description Language (IEEE Std 1364-2005).
- Siemens EDA (Mentor Graphics). ModelSim User's Manual.
- Material y apuntes del curso de Arquitectura de Computadoras, Centro Universitario de Ciencias Exactas e Ingenierías (CUCEI), Universidad de Guadalajara, periodo Verano 2026.